

Programming Basics 1



09: Polymorphie

Sommersemester 2025

Simon Dewert

HAW Hamburg

Fakultät Design, Medien und Information

E52, Finkenau 35, 22081 Hamburg

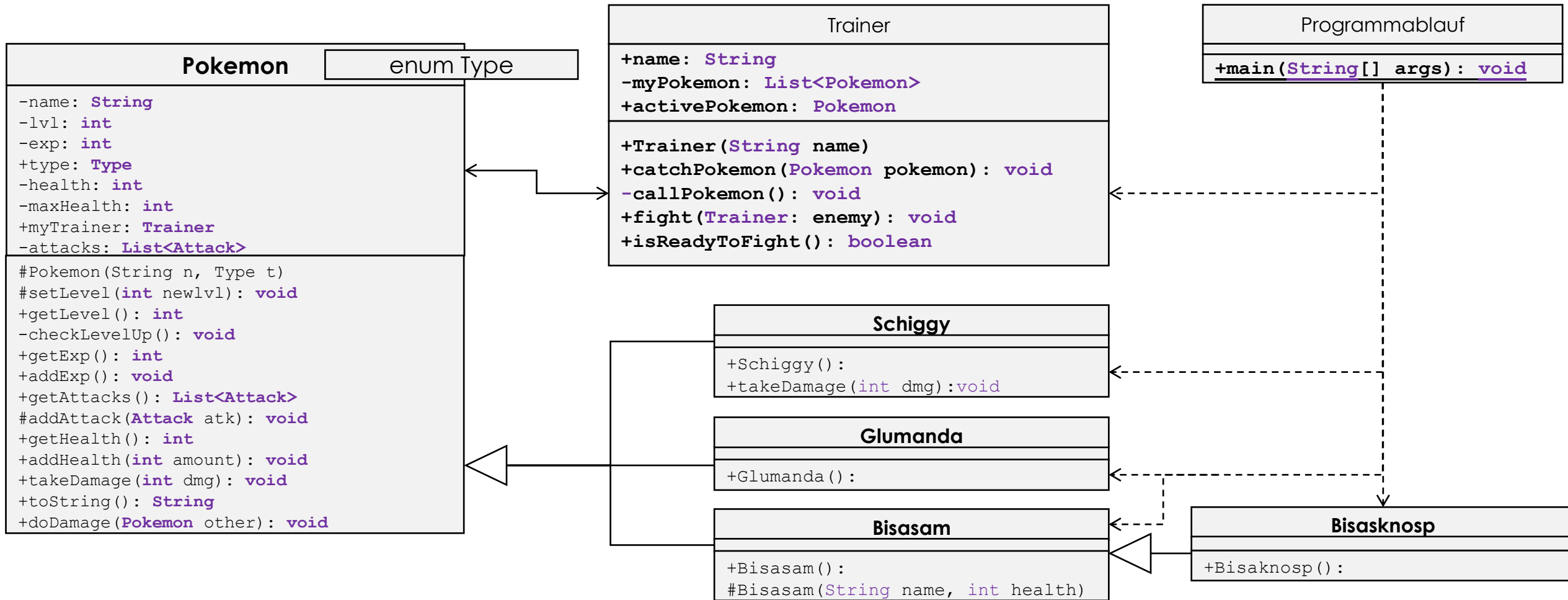
simon.dewert@haw-hamburg.de

Übersicht dieser Lehreinheit

1. Bisherige Klassenhierarchie
2. Polymorphe Variablen
3. Polymorphe Methoden
4. Generische Programmierung
 - a. Basisklassen-Arrays
 - b. Basisklassen-Parameter
 - c. Basisklassen-Rückgabetypen
5. Typidentifizierung zur Laufzeit

Klassendiagramm Pokemon

Angepasstes Klassendiagramm zum herunter geladenen Projekt



Aufgabe 9.1

25 min



Als nächstes soll unser Angriffssystem überarbeitet werden.

Wir wollen Angriffe je nach Typ mehr oder weniger effektiv werden lassen.

Dafür bitte den **Code der folgenden Seite** in der Klasse **Pokemon** einbauen.

Jede Attacke erhält jetzt eine Methode **calcDamage()**, welche den Typ des Gegnerpokemon überprüft und den angerichteten Schaden zurück liefert.

Verwenden Sie diese neue Methode in der **doDamage()** Methode von **Pokemon**.

spilettipps.de

Typ des verteidigenden Pokémon

Attacker des angreifenden Pokémon

	NORMAL	FEUER	WASSER	PFLANZE	ELEKTRO	EIS	KAMPF	GIFT	BODEN	FLUG	PSYCHO	KÄFER	GESTEIN	GEIST	DRACHE	UNLICHT	STAHL	PEE
NORMAL														×				
FEUER		-	-	+		+						+	-		-		+	
WASSER		+	-	-					+				+		-			
PFLANZE		-	+	-								-	+		-		-	
ELEKTRO			+		-				×	+					-			
EIS		-	-	+		-			+	+					+		-	
KAMPF	+					+		-		-	-	-	+	×		+	+	-
GIFT				+				-	-				-	-			×	+
BODEN		+		-	+				+	×		-	+				+	
FLUG				+	-		+					+	-				-	
PSYCHO							+	+			-				×	-		
KÄFER		-		+			-	-		-	+				-	+	-	-
GESTEIN		+				+	-		-	+		+					-	
GEIST	×										+		+		-			
DRACHE															+		-	×
UNLICHT							-				+		+			-	-	-
STAHL		-	-			+						+		+			-	+
PEE		-					+	-							+	+	-	

+ Doppelter Schaden
 Normaler Schaden
 - Halber Schaden
 × Kein Schaden

Code - Aufgabe 9.1

25 min



a) Den Markierten Code in von **Pokemon** einfügen.
Füllen Sie bitte die Lücken mit sinnvollem Code auf!

```
public class Attack {
    protected String name;
    protected int damage;
    protected Type type;
    protected List<Type> effective = new ArrayList<Type>();
    protected List<Type> resistant = new ArrayList<Type>();

    public Attack(String name, int damage, Type type) {
        this.name = name;
        this.damage = damage;
        this.type = type;
    }

    public int calcDamage(Pokemon other)
    {
        // Lokale Variable für Schadenswert die modifiziert werden darf.
        int calcDamage = damage;

        // hier: Logik für Effektivität

        // Textausgaben Bsp:
        // Rankenhieb ist NICHT effektiv!
        // Glut ist SUPER effektiv!

        return calcDamage;
    }

    public String toString() {
        return name + "(" + damage + " " + type + ")";
    }
}
```

b) Methode **doDamage()** in **Pokemon**

```
// das Pokemon greift ein anderes an und verursacht ihm Schaden.
public void doDamage(Pokemon other) {
    // es ist immer gut zu überprüfen ob ein Objekt auch wirklich existiert, bevor
    // wir es verwenden.
    if (other != null) {
        if (health > 0 && other.health > 0) {
            // Wähle zufällige Attacke aus der Liste
            Random rnd = new Random();
            // zufälliger Index zu einer Attacke aus der Liste myAttacks.
            int rndIndex = rnd.nextInt(myAttacks.size());

            // halte Referenz auf die zufällig ausgewählte Attacke
            Attack rndAttack = myAttacks.get(rndIndex);

            System.out.println(name + " greift " + other.getName() + " mit " + rndAttack + "an!");

            // übertrage den Schaden der Attacke an das angegriffene Pokemon.
            other.takeDamage(rndAttack.damage);

            // Wenn das Pokemon den Gegner besiegt hat bekommt es Erfahrung.
            if (other.getHealth() <= 0)
            {
                // Höhere Gegner liefern mehr Erfahrung.
                addExp(other.getLevel());
            }
        }
    }
}
```

Polymorphie



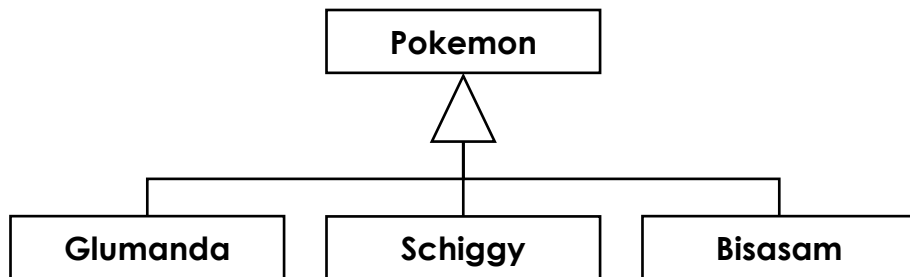
Beispiel: Unterschiedliche Gestalten des Flüssigmetallroboters T-1000 aus dem Film "Terminator 2" (1991)

Polymorphie (griechisch *polymorphia* "Vielgestaltigkeit") beschreibt die Fähigkeit eines Individuums, **unterschiedliche Gestalten** anzunehmen.

Die Grundtechnik sind die Definition von polymorphen Methoden und die Arbeit mit polymorphen Variablen. Gemeinsam erschließen diese dem Programmierer ein mächtiges neues Konzept.

Polymorphe Variablen

Eine polymorphe Variable verweist im Laufe des Programms auf **Objekte unterschiedlichen Typs**. Dies ist möglich, wenn es sich bei diesen Objekten um **Instanzen von Ableitungen der Klasse** der Variablen handelt:



Polymorphe
Variable

```
Pokemon myPokemon = new Pokemon();
```

```
myPokemon = new Glumanda();
```

```
myPokemon = new Schiggy();
```

```
myPokemon = new Bisasam();
```

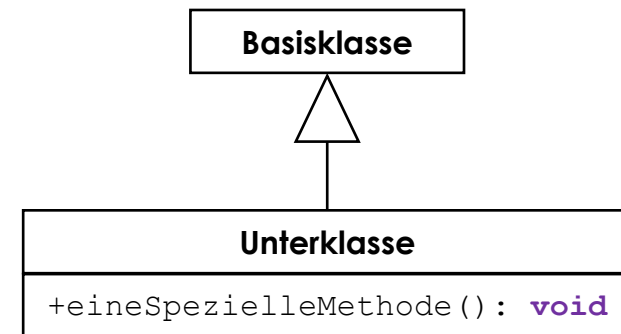
Polymorphe Variablen: Downcast

Mit einer polymorphen Variablen kann man **nicht** auf Elemente zugreifen, die exklusiv in der Unterklasse definiert wurden.

Allerdings kann eine polymorphe Variable auf den Typ der Unterklasse "heruntergecastet" werden. Diese Typumwandlung wird **Downcast** genannt:

```
Basisklasse einObjekt;  
  
einObjekt = new Unterklasse();  
  
// einObjekt.eineSpezielleMethode(); Nicht möglich!  
  
(Unterklasse) einObjekt.eineSpezielleMethode();
```

Downcast



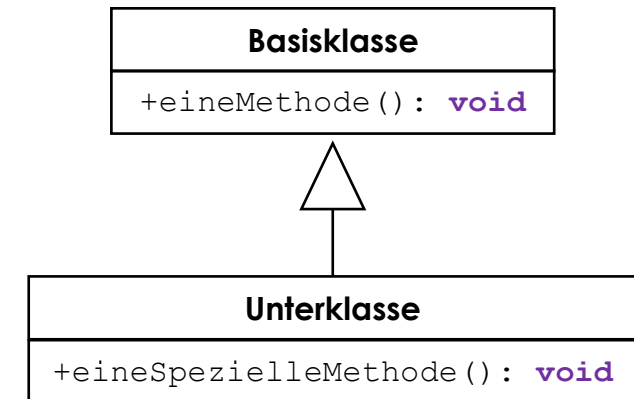
Vorsicht: Downcast funktioniert nur wenn das Objekt wirklich vom Typ Unterklasse ist!

Polymorphe Variablen: Upcast

Eine Variable einer Unterklasse kann auf den Typ einer direkten oder indirekten Basisklasse "hochgecastet" werden. Diese Typumwandlung wird **Upcast** genannt:

```
Unterklasse einObjekt = new Unterklasse();  
  
einObjekt.eineMethode();  
einObjekt.eineSpezielleMethode();  
  
((Basisklasse) einObjekt).eineMethode();  
  
// ((Basisklasse) einObjekt).eineSpezielleMethode();
```

Nicht möglich!



Durch einen Upcast kann man **nicht mehr** auf Elemente zugreifen, die exklusiv in der Unterklasse definierte wurden, jedoch bleibt das Objekt selber mitsamt seinen Elementen erhalten.

Polymorphe Methoden

(**Überschreiben** der Basisklassenmethode)

Wenn die Methode einer Basisklasse von Unterklassen verdeckt* und dadurch **individuell** implementiert wird, so handelt es sich dadurch um eine **polymorphe Methode**:

```
class Basisklasse {  
    public void eineMethode()  
    { [...] }  
}  
  
class UnterklasseA extends Basisklasse {  
    public void eineMethode()  
    { [Implementierung UnterklasseA] }  
}  
  
class UnterklasseB extends Basisklasse {  
    public void eineMethode()  
    { [Implementierung UnterklasseB] }  
}
```

*Vgl. LE08

Polymorphe Methoden: Aufruf

Generell wird beim Aufruf einer polymorphen Methode die **Version der Klasse des referenzierten Objekts** (und nicht jene der Klasse der Referenz) aufgerufen:

```
class Basisklasse {  
    public void eineMethode()  
    { System.out.println("Basisversion"); }  
}  
  
class Unterklasse extends Basisklasse {  
    public void eineMethode()  
    { System.out.println("Abgeleiteteversion"); }  
}
```

```
Basisklasse objekt = new Unterklasse();  
objekt.eineMethode();  
// Ausgabe: "Abgeleiteteversion"
```

Dadurch verhalten sich Methoden in Java – im Gegensatz zu anderen Programmiersprachen – **grundsätzlich polymorph**.

Aufgabe 9.2

10 min



Erzeugen Sie Varianten der Klasse **Attack** innerhalb der Klasse **Pokemon**.

```
public class Plant_Attack extends Attack
{
    public Plant_Attack(String name, int damage) {
        super(name, damage, Type.PFLANZE);
        effective.add(
        resistant.add(
    }
}
```

? - Siehe Tabelle
+ **effective** - **resistant**

Äquivalent sollen **Attack_Fire** und **Attack_Water** eingefügt werden.
Entnehmen Sie die Logik für die Effektivität der Tabelle rechts.

spilettipps.de

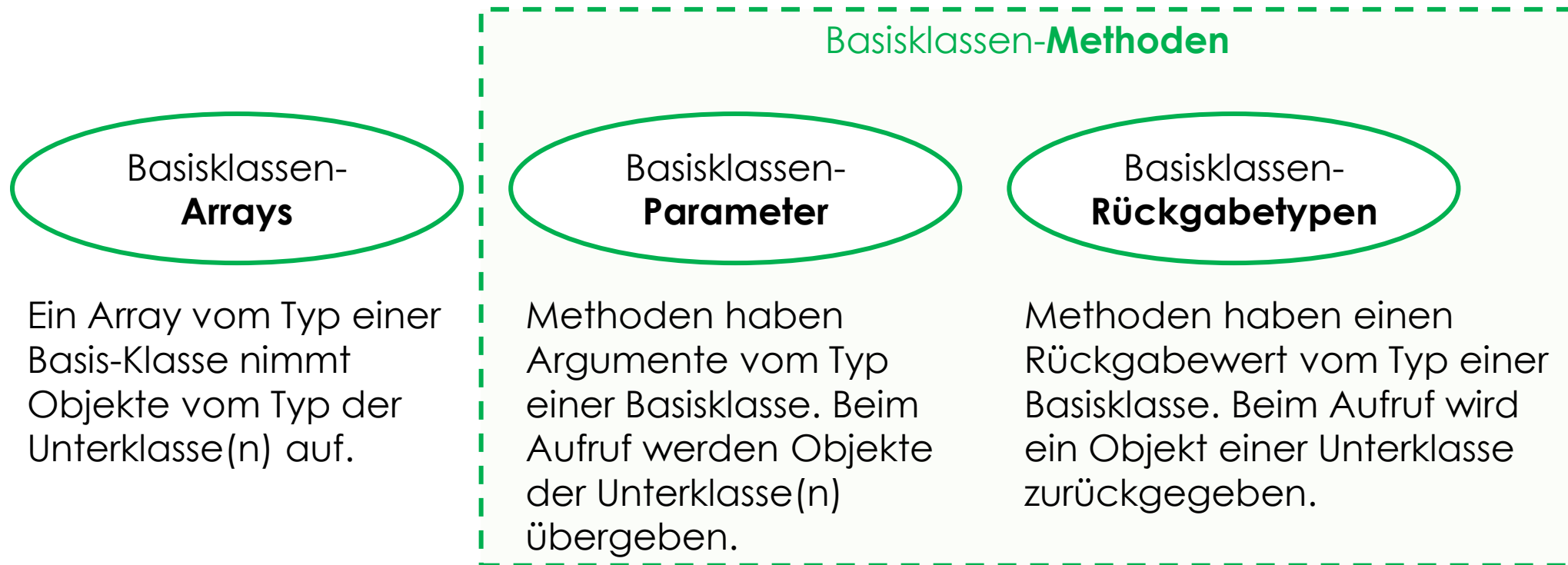
Typ des verteidigenden Pokémon

	NORMAL	FEUER	WASSER	PFLANZE	ELEKTRO	EIS	KAMPF	GIFT	BODEN	FLUG	PSYCHO	KÄFER	GESTEIN	GEIST	DRACHE	UNLICHT	STAHL	FE
NORMAL																		
FEUER		-	-	+	+							+	-		-		+	
WASSER		+	-	-					+			+	+		-			
PFLANZE		-	+	-				-	+	-		-	+		-		-	
ELEKTRO		+	-	-					×	+					-			
EIS		-	-	+		-			+	+					+		-	
KAMPF	+				+		-		-	-	-	+	×		+	+	-	
GIFT				+			-	-				-	-				×	+
BODEN		+		-	+		+		×			-	+				+	
FLUG				+	-	+						+	-				-	
PSYCHO						+	+			-					×	-		
KÄFER		-		+			-	-		-	+				+	-	-	
GESTEIN		+			+	-	-		+	+		+					-	
GEIST	×									+			+		-			
DRACHE														+		-	×	
UNLICHT							-			+		+		+		-	-	
STAHL		-	-		+							+					-	+
FE		-				+	-							+	+	-		

Legende: + Doppelter Schaden, Normaler Schaden, - Halber Schaden, × Kein Schaden

Polymorphie und generische Programmierung

Polymorphie ist die **Voraussetzung** für sogenannte *generische* Programmierung, bei der drei typische Ausprägungen unterschieden werden, die üblicherweise miteinander kombiniert werden:



Basisklassen-Arrays

siehe **Aufgabe 9.1**

Ein Array, dessen Elemente vom Typ einer Basisklasse sind, kann **Objekte verschiedener Unterklassen** aufnehmen:

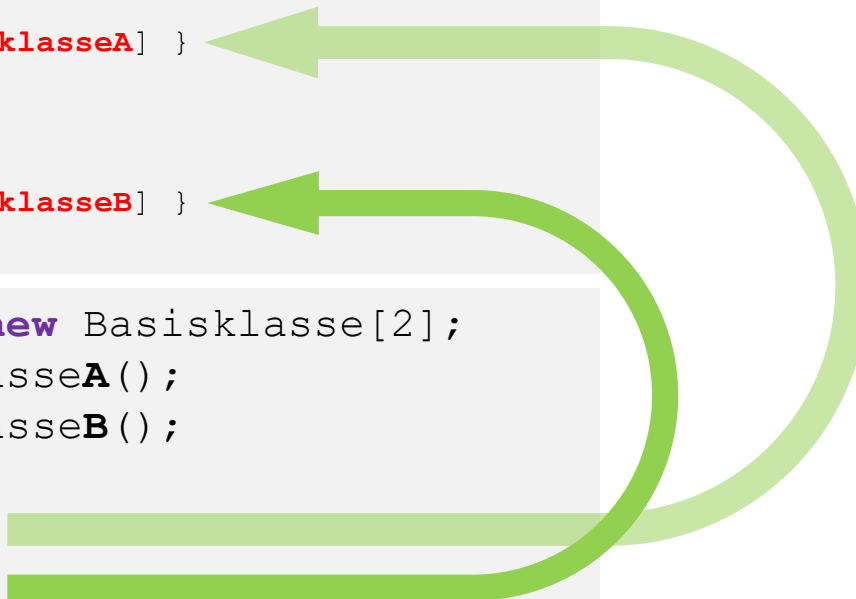
```
class Basisklasse {  
    public eineMethode() { [...] }  
}
```

```
class UnterklasseA extends Basisklasse {  
    public eineMethode() { [Implementierung UnterklasseA] }  
}
```

```
class UnterklasseB extends Basisklasse {  
    public eineMethode() { [Implementierung UnterklasseB] }  
}
```

```
Basisklasse[] basisKlassenArray = new Basisklasse[2];  
basisKlassenArray[0] = new UnterklasseA();  
basisKlassenArray[1] = new UnterklasseB();
```

```
basisKlassenArray[0].eineMethode();  
basisKlassenArray[1].eineMethode();
```



Basisklassen-Methoden

Übergabewert Polymorph

Basisklassen-Parameter

Methoden, deren Parameter vom Typ einer Basisklasse sind, werden Objekte verschiedener Unterklassen übergeben:

```
class Basis {}  
class Unter extends Basis {}  
  
class Andere {  
    void methode(Basis basisParam) {...}  
}
```

```
Andere anderesObj = new Andere();  
Unter unterObjekt = new Unter();  
  
anderesObj.methode(unterObjekt);
```

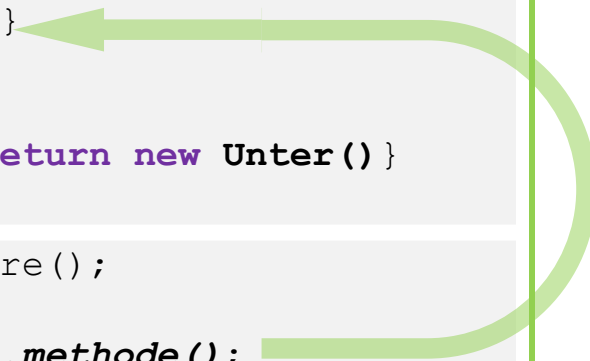
Rückgabewert Polymorph

Basisklassen-Rückgabetypen

Methoden mit Rückgabewert vom Typ einer Basisklasse können Objekte verschiedener Unterklassen zurückgeben:

```
class Basis  
{ public void methode() {...}}  
class Unter extends Basis  
{ public void methode() {...} }  
  
class Andere {  
    Basis methodeMitRueck() {return new Unter()}  
}
```

```
Andere anderesObj = new Andere();  
  
anderesObj.methodeMitRueck().methode();
```



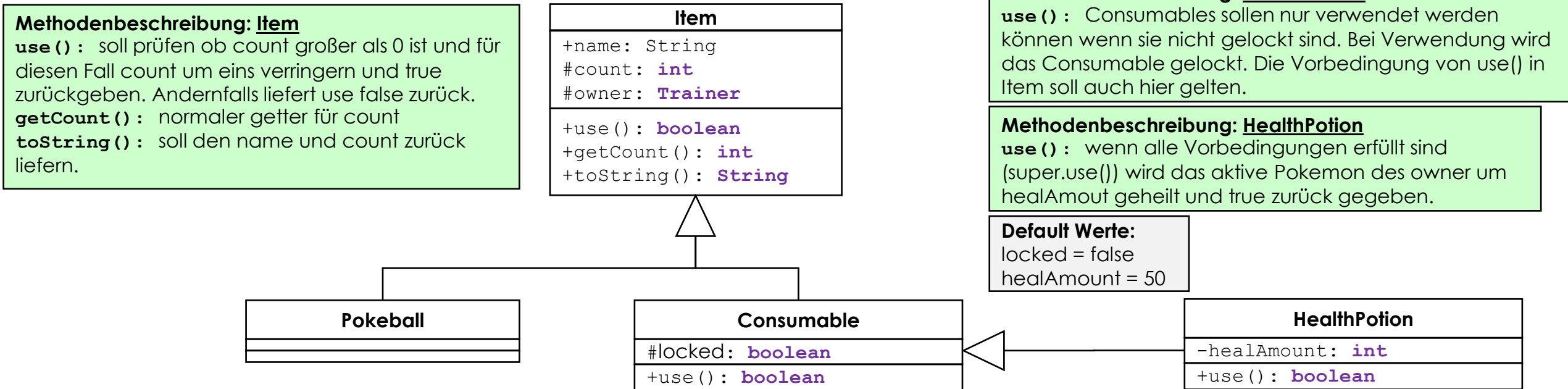
Aufgabe 9.3

20 min



Erzeugen Sie die folgenden **Klassen aus dem UML-Diagramm** im neuen **Paket Item**, nach folgendem Klassendiagramm.

Geben Sie der Klasse **Trainer** ein neues Attribut Item: `public Item inventory;`



Typidentifizierung: `instanceof`

Beim **Downcast** wird eine polymorphe Variable auf den Typ einer Unterklasse "heruntergecastet" (vgl. Folie 7). Dies gelingt aber nur, sofern es sich bei dem referenzierten Objekt tatsächlich um ein **Objekt der Unterklasse** handelt.

Um sicherzustellen, dass ein bestimmtes Objekt eine **Instanz einer bestimmten Klasse** ist, kann der Operator `instanceof` verwendet werden:

```
class Basisklasse {}  
  
class Unterklasse extends Basisklasse {}  
  
class Irgendeineklasse {}
```

```
Basisklasse object = new Unterklasse();  
  
if (object instanceof Basisklasse) {...} true  
if (object instanceof Unterklasse) {...} true  
if (object instanceof Irgendeineklasse) {...} false
```

Typidentifizierung: `getClass()`

Alternativ zu `instanceof` kann die in `java.lang.Object` (und damit in jeder Klasse) definierte Methode `getClass()` in Verbindung mit einer speziellen Syntax verwendet werden:

```
Basisklasse obj = new Unterklasse();  
  
if (obj.getClass() == Unterklasse.class) {...} true
```

`KLASSENNAME.class`



Die Methode `getClass()` ist jedoch vor allem dazu geeignet, **zwei Objekte auf Klassengleichheit** zu prüfen:

```
class KlasseA {}  
  
class KlasseB {}  
  
KlasseA objEins = new KlasseA();  
KlasseA objZwei = new KlasseA();  
KlasseB objDrei = new KlasseB();  
  
if (objEins.getClass() == objZwei.getClass()) {...} true  
if (objEins.getClass() == objDrei.getClass()) {...} false
```

getClass() VS. instanceof

Typidentifizierung

Mit **getClass()** lassen sich die Klassen direkt vergleichen, während bei **instanceof** auch Basisklassen miteinbezogen werden.

```
class Basisklasse {}  
  
class Unterklasse extends Basisklasse {}  
  
class IrgendeineKlasse {}
```

```
Basisklasse object = new Unterklasse();
```

```
true  
true  
false  
if (object instanceof Basisklasse) {...}  
if (object instanceof Unterklasse) {...}  
if (object instanceof IrgendeineKlasse) {...}
```

```
Basisklasse object = new Unterklasse();
```

```
if (object.getClass() == Basisklasse.class) {...}  
if (object.getClass() == Unterklasse.class) {...}  
if (object.getClass() == IrgendeineKlasse.class) {...}
```

```
false  
true  
false
```

Aufgabe 9.4

5 min



Die Methode `catchPokemon()` des **Trainers** soll jetzt nur ein **Pokemon** fangen können, wenn der **Trainer** noch Gegenstände vom Typ **Pokeball** hat.

Jeder **Trainer** soll von Beginn an (im Konstruktor) einen **Pokeball** bekommen.

Aufgabe 9.4

5 min



Die Methode **catchPokemon()** des **Trainers** soll jetzt nur ein **Pokemon** fangen können, wenn der **Trainer** noch Gegenstände vom Typ **Pokeball** hat.

Jeder **Trainer** soll von Beginn an (im Konstruktor) einen **Pokeball** bekommen.

Viele Dank für Ihre Aufmerksamkeit



Hausaufgabe 9

ca. 60 min



Erzeugen Sie eine neue Klasse **Inventory** (im Package **Item**).
Diese Klasse soll **Item inventory** der **Trainer** ersetzen.

Inventory
-owner: Trainer -items: List<Item> -maxSize: int
+getItemCount(): int +search(String name): Item +add(Item i): void +drop(Item i): void +use(Item i): boolean +toString(): String

Infos:

- **getItemCount()** zählt die counts aller Items der Liste items
- **search()** durchsucht die Liste nach einem Gegenstand mit übergebenem Namen
- **add()** darf nur Items hinzufügen, bis die MaxSize erreicht ist.
- **drop()** entfernt einen Gegenstand aus der Liste
- **use(Item i)** verwendet/ einen Gegenstand
- **toString()** liefert eine Auflistung aller Gegenstände in items.